ELEC 475 Lab 1

Thomas Wilkinson: 20269155

Ahmed Iqbal: 20272366

Model Details

The model used in this lab was an MLP4 autoencoder. The encoder is made up of two linear fully connected layers with ReLU activation functions performed after each layer. The decoder is made up of two more linear fully connected layers with the ReLU activation function performed after the first and the sigmoid activation function performed after the second. The inputs for this model where flattened images from the MNIST dataset (28x28 = 784 Pixels). The outputs for this model where similar approximated images (28x28 = 784 Pixels). For a detailed view of the inputs and outputs of each layer, check Figure 1.

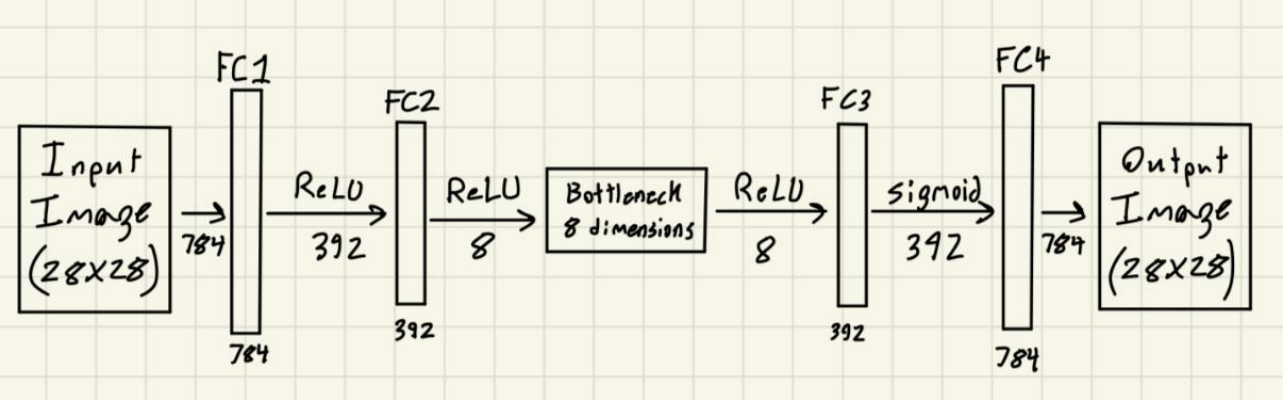

Figure 1: Architectural diagram of the MLP4 autoencoder used in this lab.

Training Details

Training can be reproduced by running "**python train.py -z 8 -e 50 -b 2048 -s MLP.8.pth -p loss.MLP.8.png**" in the command terminal. The data used for training is the images from the grayscale MNIST image dataset, which were converted to tensors before use. The model used was the same as the one outlined in the "Model Details" section of this report. The optimization method used was Adam with the hyperparameters being 1e-3 for the learning rate and 1e-5 for the weight decay. The learning rate scheduler mode was set to "min", thereby lowering the learning rate if the loss plateaus or increases. The loss function was the Mean Squared Error between the input image and the reconstructed version at the output. The initial weights for the linear layers where randomized using the Xavier uniform function and the bias was set to 0.01.

Note: The bottleneck size, epoch number, batch size, and file paths for weights and loss function were initialized when running the terminal command:

-z 8, being for the bottleneck size

-e 50, being for the number of epochs

-b 2048, being for the batch size

-s MLP.8.pth, being for the file path where the parameters (weights) are saved

-p loss.MLP.8.png, being for the file path where a plot of the loss function is saved

Results

The testing phase can be initiated by running the following command in the terminal "**python lab1.py -l MLP.8.pth**". For testing, in section 4 of the lab, the autoencoder worked as expected, reconstructing the inserted images with high accuracy. Even though the reconstructed images where slightly blurrier than the inserted images, as seen in Figure 3, they still held the same underlying shape. For stage 5, image denoising, the model performed better than expected. As it can be seen in Figure 4, the noisy image of a 3 was successfully recreated at the output. For stage 6, the bottleneck interpolation worked significantly well (check Figure 5), forming good transitions between the entered digits. As seen in Figure 2, the loss function decreased significantly during the first 10 epochs, then started to plateau as it got closer to 50 epochs. This is to be expected as this is what should happen when the loss gets closer to its minimum.

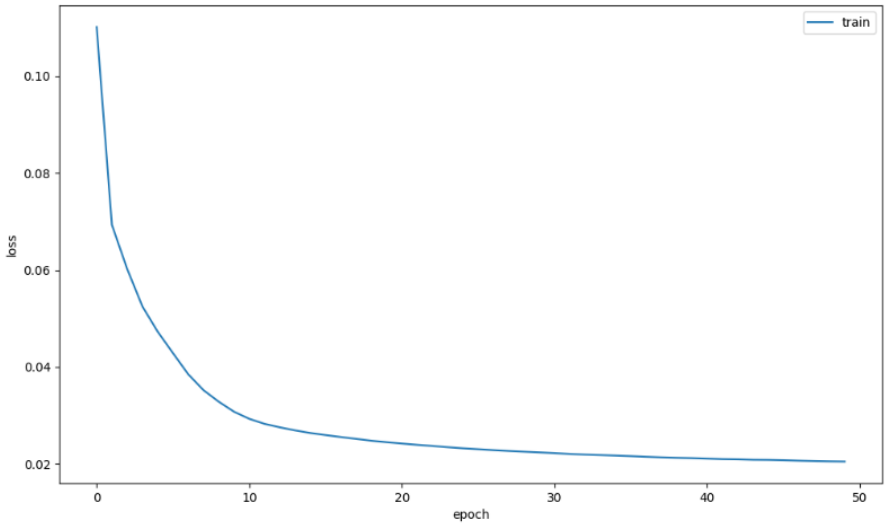

Figure 2: Plot of loss function throughout the training stage of the model.

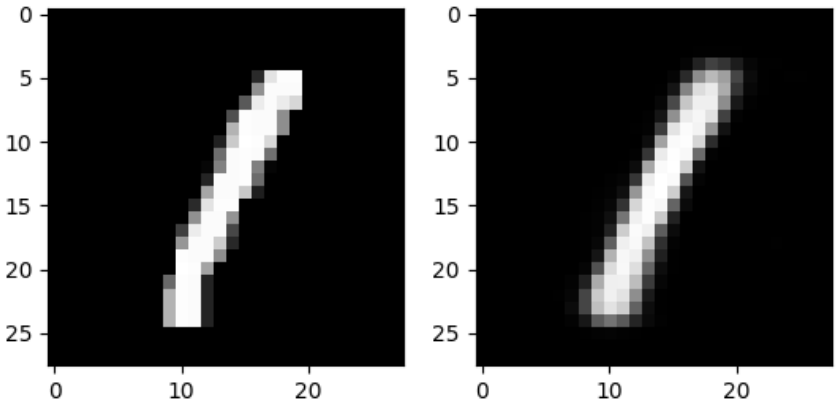

Figure 3: Image depicting inserted digit on the left and reconstructed output on the right for testing stage of the autoencoder.

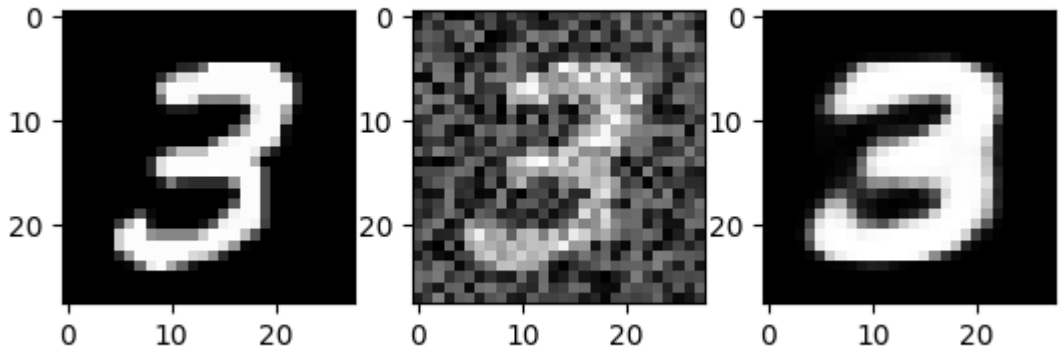

Figure 4: Image depicting successful output for image denoising. Normal digit can be seen on the left, noisy image used as an input can be seen in the middle, and reconstructed output can be seen on the right.

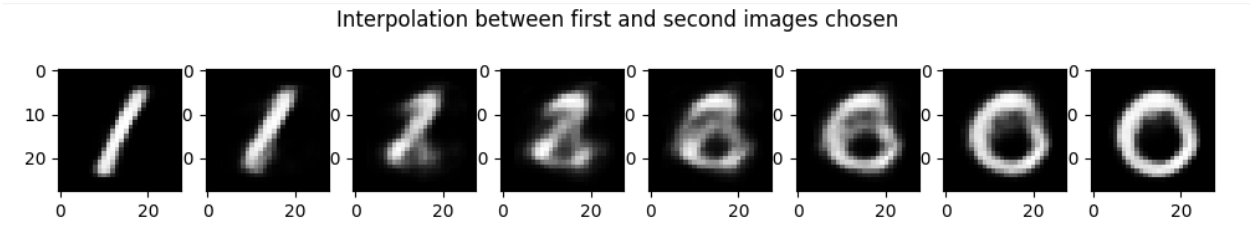

Figure 5: Image depicting the output from the interpolation of two digits.